42. Kubernetes-Oriented Go: Controllers and client-go

42. Kubernetes-Oriented Go: Controllers and client-go

Phase: Phase 6 — Systems and infrastructure
Estimated time: 2–3 days
Slug: `42-kubernetes-go`

Learning objectives

- Explain control loops (reconcile)
- Sketch a controller with client-go concepts
- Use leader election awareness
- Design for level-triggered reconciliation

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Kubernetes controllers reconcile desired vs actual state. Go and client-go dominate this ecosystem.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Reconcile loops: read state → diff → act → repeat.
- Informers/caches reduce API server load.
- Idempotent actions: safe to retry.
- Status subresources report progress.
- Leader election for active/passive controllers.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-kubernetes-o && cd /tmp/gw-kubernetes-o
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import "fmt"

func Reconcile(desired, actual int) string {
    if actual == desired {
        return "ok"
    }
    return "update"
}

func main() { fmt.Println(Reconcile(3, 1)) }
```

Run it:

```
go run .
```

Expected output:

```
update
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Write a pure reconcile decision function.
2. List what makes an action idempotent.
3. Sketch an informer event → queue → worker.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Control loop

Kubernetes controllers observe desired state (spec) and drive actual state toward it — continuously.

```
for {
  desired := getSpec()
  actual := getStatus()
  if actual != desired { act() }
}
```

client-go concepts

Informers/caches reduce API spam; workqueues debounce; reconcile must be idempotent.

Operators

Encode domain ops knowledge as controllers. Go dominates this ecosystem.

You do not need a cluster for the exercise — implement a tiny reconcile function over an in-memory store.

Failure modes to watch while coding

- One-shot scripts instead of reconcile loops
- Non-idempotent reconciles
- Hot-looping without backoff

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- Operators exist in other langs; Go is the default for k8s.
- Same reconcile idea as config management loops.
- Watch API vs poll — prefer events with resync.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

Going deeper

Never block forever in reconcile without context; requeue with backoff.

Practice checklist

Before the exercise, confirm you can:

1. Explain "Kubernetes-Oriented Go: Controllers and client-go" aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- One-shot scripts instead of reconcile loops
- Non-idempotent reconciles
- Hot-looping without backoff

Exercises

Work in `exercises/42-kubernetes-go/`. Read that folder's `README.md` before you code.

What you will build

Implement reconcile decision stubs matching tests.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/42-kubernetes-go/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. What is reconcile?
2. Why idempotent?
3. Role of informers?

Further reading

- https://github.com/kubernetes/client-go